INTERVIEW PREPARATION TEST

JAVA DSA BATCH — CAPSTONE GRAND ASSESSMENT

Batch: PW Java Classes (Java DSA)	**Duration:** 3 Hours (180 Mins)
Total Marks: 100 Marks (MCQ: 40, Coding: 60)	**Assessment Type:** Interview Preparation Test (Grand Contest)

CRITICAL CANDIDATE INSTRUCTIONS & SUBMISSION PROTOCOL

1. Section A (MCQs 1 to 20 — 40 Marks): Each question carries 2 marks. There is a negative marking of -0.5 marks for incorrect answers. **Submission Requirement:** You MUST submit a separate written document (PDF/Word/Markdown) titled MCQ_Answers_<YourName>_<RollNo>.pdf containing your chosen option (A/B/C/D) along with a concise 1-2 sentence technical justification for each answer.

2. Section B (Coding Problems 1 to 6 — 60 Marks): Each problem carries 10 marks. Code MUST be written in clean, production-grade Java (Java 8+). **Submission Requirement:** You MUST submit your solutions via **GitHub**. Create a public/private GitHub repository named java-dsa-interview-test-<yourname>, organize each solution into dedicated packages/files (e.g., Problem1.java ... Problem6.java), and include a detailed README.md explaining your approach, Time Complexity, and Space Complexity. Submit the repository link on the portal.

3. Code Integrity: Submissions with generic brute-force solutions exceeding problem time limits (TLE) will not receive full credit. Solutions must strictly adhere to the specified asymptotic bounds.

SECTION A: MULTIPLE CHOICE QUESTIONS (20 Questions × 2 Marks = 40 Marks)

Select the single most optimal choice. Submit answers in your MCQ document with reasoning.

Q1. Consider the following Java code snippet. What will be printed to standard output?

```
String s1 = "Java";
String s2 = new String("Java");
String s3 = s2.intern();
System.out.println((s1 == s2) + " " + (s1 == s3) + " " + s1.equals(s2));
```

A) true true true

B) false true true

C) false false true

D) true false true

Q2. What is the tightest worst-case Time Complexity of the following recursive function for an integer input N?

```
void solve(int n) {
    if (n <= 1) return;
    for (int i = 1; i <= n; i++) { System.out.print(i); }
    solve(n / 2);
    solve(n / 2);
}
```

A) O(N)

B) O(N log N)

C) $O(N^2)$

D) O(2^N)

Q3. In bit manipulation, what does the expression (n & (n - 1)) == 0 evaluate to for any positive integer n > 0?

A) Checks if n is an odd number

B) Checks if n is a prime number

C) Checks if n is a power of 2

D) Clears the lowest set bit of n

Q4. Given a sorted array of N integers, we want to find if there exists a pair with sum equal to K. Which algorithmic technique achieves this in O(N) time and O(1) auxiliary space?

A) Binary Search on each element — O(N log N) time, O(1) space

B) Two Pointers (left at 0, right at N-1) converging inwards — O(N) time, O(1) space

C) HashSet lookup — O(N) time, O(N) space

D) Nested loop brute force — $O(N^2)$ time, O(1) space

Q5. To rotate an N × N matrix clockwise by 90 degrees in-place without allocating a new matrix, what sequence of transformations is required?

A) Reverse each row, then transpose the matrix

B) Transpose the matrix, then reverse each row

C) Transpose the matrix, then reverse each column

D) Reverse each column, then reverse each row

Q6. In Java 8 and later, what happens internally when the number of elements in a single bucket of a HashMap exceeds the treeification threshold (TREEIFY_THRESHOLD = 8) and total capacity is at least 64?

A) The bucket immediately throws a ConcurrentModificationException

B) The bucket converts its internal LinkedList into a Red-Black Tree (TreeNode), reducing search time from O(N) to O(log N)

C) The HashMap drops colliding keys and retains only the most recent entry

D) The capacity doubles but the bucket remains an O(N) singly linked list

Q7. Which of the following sorting algorithms is stable, has worst-case time complexity of O(N log N), but requires O(N) auxiliary memory?

A) Quick Sort

B) Heap Sort

C) Merge Sort

D) Selection Sort

Q8. In the 'Search on Answer Space' paradigm (e.g., Koko Eating Bananas, Capacity to Ship Packages), what essential mathematical property must the feasibility function canComplete(speed) satisfy across the search range?

A) It must be strictly quadratic

B) It must be monotonic (if speed X is feasible, every speed Y > X is also guaranteed feasible)

C) It must produce prime values

D) It must run in O(1) time without reading the input array

Q9. In Floyd's Cycle Detection Algorithm (Tortoise and Hare), suppose a cycle of length C exists. When slow enters the cycle, fast is already inside. Why are they mathematically guaranteed to meet?

A) Fast moves 2 steps and slow moves 1 step, decreasing the relative gap between them by exactly 1 in every iteration until gap becomes 0 modulo C

B) Fast reverses direction upon hitting the tail of the list

C) The call stack forces both pointers to reset to head

D) Slow accelerates to speed 2 after traversing half the list

Q10. What is the total amortized time complexity of finding the 'Next Greater Element' for all N elements in an array using a Monotonic Decreasing Stack?

A) $O(N^2)$ because of the while loop inside the for loop

B) O(N log N) because elements are maintained in sorted order

C) O(N) because every element is pushed onto the stack exactly once and popped at most once

D) O(1) auxiliary space and O(N) time

Q11. To solve the 'Sliding Window Maximum' problem for an array of size N with window size K in optimal O(N) time, which data structure is used and what invariant is maintained?

A) Standard FIFO Queue maintaining elements in arrival order

B) Monotonic Deque (ArrayDeque) storing indices with values in strictly decreasing order

C) Min-Heap of size K storing indices

D) Stack storing maximum element seen so far

Q12. If the Inorder traversal of a binary tree visits nodes in sorted ascending order, which of the following statements MUST be true?

A) The tree is a Complete Binary Tree

B) The tree is a valid Binary Search Tree (BST)

C) The tree has minimum possible height

D) The Postorder traversal will also be sorted

Q13. When deleting a node with two children from a Binary Search Tree, what is the standard replacement node to maintain the BST invariant?

A) The root of the whole tree

B) Either the Inorder Predecessor (maximum of left subtree) or Inorder Successor (minimum of right subtree)

C) The left child's immediate left leaf

D) Any arbitrary node with 0 children

Q14. What is the optimal time complexity to find the K-th Largest element in an unsorted stream of N elements using a Heap?

A) O(N log N) by inserting all elements into a Max-Heap

B) O(N log K) by maintaining a Min-Heap of bounded size K

C) O(K log N) by building an array and using Selection Sort

D) O(N) space and $O(N^2)$ time

Q15. Why does Breadth-First Search (BFS) guarantee finding the shortest path between two nodes in an unweighted graph, whereas Depth-First Search (DFS) does not?

A) BFS visits nodes radially layer-by-layer; any path of length L is completely evaluated before discovering paths of length L+1

B) BFS uses recursion which minimizes call stack depth

C) BFS sorts edge weights dynamically using a PriorityQueue

D) DFS cannot handle graphs with cycles even with a visited array

Q16. Which of the following algorithm choices is correct when finding single-source shortest paths in a directed graph that contains negative edge weights but no negative weight cycles?

A) Dijkstra's Algorithm using a Min-Heap — O(E log V)

B) Standard BFS using a Queue — O(V + E)

C) Bellman-Ford Algorithm — O(V × E)

D) Prim's Algorithm — O(E log V)

Q17. In Kahn's Algorithm for Topological Sort on a directed graph with V vertices, what indicates the presence of a directed cycle?

A) The in-degree of all vertices becomes zero immediately

B) The number of vertices processed from the Queue is strictly less than V

C) The Queue overflows its maximum capacity

D) The adjacency list contains disconnected components

Q18. Consider the Coin Change problem with coin denominations {1, 3, 4} and target amount 6. What do the Greedy approach and the optimal Dynamic Programming approach produce?

A) Greedy: 2 coins {3, 3}; DP: 3 coins {4, 1, 1}

B) Greedy: 3 coins {4, 1, 1}; DP: 2 coins {3, 3}

C) Both Greedy and DP produce 2 coins {3, 3}

D) Both Greedy and DP fail because coin change is unsolvable

Q19. In the Longest Common Subsequence (LCS) problem for strings S1 (length M) and S2 (length N), what is the state transition when characters S1[i-1] != S2[j-1]?

A) dp[i][j] = 1 + dp[i-1][j-1]

B) dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1])

C) dp[i][j] = Math.min(dp[i-1][j], dp[i][j-1]) + 1

D) dp[i][j] = 0

Q20. When generating all Subsets of an array using the backtracking pattern, what happens if the developer writes result.add(tempList); instead of result.add(new ArrayList<>(tempList));?

A) The code throws a CompileTimeTypeMismatchException

B) It creates an infinite recursive loop

C) The result list ends up containing references to the same mutable list object, which ends up empty [[], [], ...] after un-choosing

D) Java automatically deep-copies the list during List.add()

SECTION B: CODING CONTEST PROBLEMS (6 Problems × 10 Marks = 60 Marks)

All solutions must be implemented in Java (Java 8+). Submit source code repository link via GitHub. Include detailed time and space complexity explanations in your README.

Problem 1: Subarray Sum Equals K | *10 Marks*

Given an integer array nums (which may contain both positive and negative integers) and an integer k, return the total number of continuous subarrays whose sum equals k.
Constraint: Your solution must run in **O(N)** time complexity.

Constraints & Complexity:
• 1 ≤ nums.length ≤ 2 × 10■
• -1000 ≤ nums[i] ≤ 1000
• -10■ ≤ k ≤ 10■
• Expected Time Complexity: O(N)
• Expected Space Complexity: O(N)

Sample Input 1:	`nums = [1, 1, 1], k = 2`
Sample Output 1:	`2` `Explanation: Subarrays [1, 1] at indices [0, 1] and [1, 2] both sum to 2.`
Sample Input 2:	`nums = [1, 2, 3, -2, 1], k = 3`
Sample Output 2:	`3` `Explanation: Subarrays [1, 2], [3], and [2, 3, -2] all sum to 3.`

Problem 2: Linked List Cycle II | *10 Marks*

Given the head of a singly linked list, return the node where the cycle begins. If there is no cycle, return null.
A cycle exists if there is some node in the list that can be reached again by continuously following the next pointer.
Mandate: You MUST solve this without modifying the linked list and using only **O(1) auxiliary memory**. Solutions using extra collections (like HashSet) for node tracking will not receive full credit.

Constraints & Complexity:
• Number of nodes is in range [0, 10■]
• -10■ ≤ Node.val ≤ 10■
• Expected Time Complexity: O(N)
• Expected Auxiliary Space: O(1)

Sample Input 1:	`head = [3, 2, 0, -4], pos = 1 (tail connects to node index 1 with value 2)`
Sample Output 1:	`tail connects to node index 1 (returns reference to Node with value 2)`
Sample Input 2:	`head = [1, 2], pos = -1 (no cycle)`
Sample Output 2:	`null (no cycle detected)`

Problem 3: Daily Temperatures | *10 Marks*

Given an array of integers temperatures representing daily temperatures, return an array answer such that answer[i] is the number of days you have to wait after the i-th day to get a warmer temperature. If there is no future day for which this is possible, keep answer[i] = 0 instead.
Mandate: Solutions with $O(N^2)$ time complexity will exceed the execution time limit (TLE). Your solution must achieve optimal linear time complexity.

Constraints & Complexity:
• 1 ≤ temperatures.length ≤ 10■
• 30 ≤ temperatures[i] ≤ 100
• Expected Time Complexity: O(N)
• Expected Space Complexity: O(N)

Sample Input 1:	`temperatures = [73, 74, 75, 71, 69, 72, 76, 73]`
Sample Output 1:	`[1, 1, 4, 2, 1, 1, 0, 0]` `Explanation: For day 0 (73), day 1 is warmer (74), wait = 1. For day 2 (75), day 6 is warmer (76), wait = 4.`
Sample Input 2:	`temperatures = [30, 40, 50, 60]`
Sample Output 2:	`[1, 1, 1, 0]`

Problem 4: Lowest Common Ancestor of a Binary Tree | *10 Marks*

Given a binary tree, find the Lowest Common Ancestor (LCA) of two given nodes p and q.
The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).
Mandate: Implement an optimal recursive solution that visits each node at most once without maintaining external parent-pointer maps.

Constraints & Complexity:
• Number of nodes is in range [2, 10■]
• -10■ ≤ Node.val ≤ 10■
• All Node.val are unique. p and q are guaranteed to exist in the tree.
• Expected Time Complexity: O(N)
• Expected Auxiliary Space: O(H) where H is tree height

Sample Input 1:	`root = [3, 5, 1, 6, 2, 0, 8, null, null, 7, 4], p = 5, q = 1`
Sample Output 1:	`3` `Explanation: The LCA of nodes 5 and 1 is 3.`
Sample Input 2:	`root = [3, 5, 1, 6, 2, 0, 8, null, null, 7, 4], p = 5, q = 4`
Sample Output 2:	`5` `Explanation: The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself.`

Problem 5: Course Schedule II | *10 Marks*

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you must take course b_i first if you want to take course a_i.
Return the ordering of courses you should take to finish all courses. If there are multiple valid orderings, return **any** of them. If it is impossible to finish all courses (due to a dependency deadlock), return an **empty array** [].
Mandate: Your algorithm must detect dependency cycles and output a valid linear execution order in linear time.

Constraints & Complexity:
• 1 ≤ numCourses ≤ 2000
• 0 ≤ prerequisites.length ≤ numCourses × (numCourses - 1)
• prerequisites[i].length == 2
• Expected Time Complexity: O(V + E)
• Expected Space Complexity: O(V + E)

Sample Input 1:	`numCourses = 2, prerequisites = [[1, 0]]`
Sample Output 1:	`[0, 1]` `Explanation: To take course 1 you must finish course 0. Valid order: [0, 1].`
Sample Input 2:	`numCourses = 4, prerequisites = [[1,0], [2,0], [3,1], [3,2]]`
Sample Output 2:	`[0, 2, 1, 3] (or [0, 1, 2, 3])`
Sample Input 3:	`numCourses = 2, prerequisites = [[1,0], [0,1]]`
Sample Output 3:	`[]` `Explanation: Cycle exists between 0 and 1. Deadlock prevents completing all courses.`

Problem 6: Partition Equal Subset Sum | *10 Marks*

Given an integer array nums, return true if you can partition the array into two subsets such that the sum of the elements in both subsets is equal, or false otherwise.
Mandate: Exponential brute-force recursion (O(2^N)) will exceed the time limit. Your solution must run within optimal polynomial time.

Constraints & Complexity:
• 1 ≤ nums.length ≤ 200
• 1 ≤ nums[i] ≤ 100
• Expected Time Complexity: O(N × target) where target = sum / 2
• Expected Space Complexity: O(target)

Sample Input 1:	`nums = [1, 5, 11, 5]`
Sample Output 1:	`true` `Explanation: The array can be partitioned as [1, 5, 5] and [11], both summing to 11.`
Sample Input 2:	`nums = [1, 2, 3, 5]`
Sample Output 2:	`false` `Explanation: totalSum = 11 (odd). Cannot be divided into two equal integer subsets.`

End of Question Paper — Best of luck with your contest submission!